

Decomposing Algebraic Numbers into Low-Degree Roots

Exact Wolfram Language methods, global additive minimization, and certified product searches

Prepared for Vladimir Reshetnikov

September 2026

Abstract

Given an exact algebraic number, one may ask for a representation as a finite sum or product of algebraic numbers that minimizes the largest absolute degree of an individual term. The two degree-nine examples in the original Mathematica question are recovered by exact resultant calculations, and a Galois-theoretic lower bound proves that degree three is globally optimal, even when arbitrarily many terms and fields outside the input field are permitted.

For sums, we prove a finite, complete criterion: it suffices to take rational linear combinations of low-degree subset sums of the input's conjugates. This yields an exhaustive Wolfram Language algorithm without explicit Galois-group or subfield enumeration. For two specified number fields, both sum and product decomposition reduce to rational linear algebra; the product criterion is the nonvanishing of a field-space intersection.

For unrestricted products, the supplied algorithms are certified searches, not a claimed general terminating optimizer. We explain the precise obstruction to a tempting normal-closure reduction, including a degree-eight counterexample whose optimal product degree is four but whose optimal sum degree is eight. The accompanying package, native-kernel test files, and independently executed symbolic and finite-group checks separate mathematical guarantees from implementation-validation status.

Contents

1	The problem and the guarantees	2
1.1	Why a polynomial Root is not itself the degree certificate	2
2	The two examples: discovery, reconstruction, and optimality	3
2.1	Immediate Wolfram Language use	3
2.2	Recovering the cubics rather than supplying them	4
2.3	Transparent certificates inside the degree-nine fields	4
3	Resultants: exact pair detection	5
3.1	Divisibility, not equality	6
3.2	Companion matrices are the same construction	6
3.3	Coefficient matching and rationality	6
4	Global lower bounds	7
5	Two specified number fields: both operations are linear	8
5.1	The additive test	8
5.2	The product test: a nonzero intersection	8
5.3	Getting consistent exact coordinates	9
6	A complete finite algorithm for sums	9
6.1	A preliminary descent to the normal closure	9
6.2	The conjugate-subset theorem	9
6.3	Halving the subset search and bounding the number of terms	11
6.4	Algorithm and stopping certificate	11
6.5	Complexity and explicit resource limits	11
7	Changing the objective changes the answer	12
7.1	An explicit sum of two sextic roots	12
7.2	Why degree five or less cannot suffice	13
8	Products can require factors outside the normal closure	14
8.1	The polynomial and its splitting field	14
8.2	A subgroup obstruction to every smaller-degree internal factor	15
8.3	A relevant manuscript needs a correction	15
8.4	Why taking norms does not repair the reduction	15
9	Certified product searches and a practical workflow	15
9.1	Finite rational-polynomial catalogs	16
9.2	More than two factors: exact residual search	16
9.3	What exhaustive enumeration does and does not establish	16
9.4	Recommended order of computation	17
10	Implementation details, validation, and reproducibility	17
10.1	Package interfaces and output contract	17
10.2	Executed independent checks	18
10.3	Native Wolfram execution status	18
11	Conclusion	19
A	Complete Wolfram Language package	20
B	Independent verification design	24

1 The problem and the guarantees

Let $\alpha \in \overline{\mathbb{Q}}$ be specified exactly. Write

$$\text{adeg}(\beta) = [\mathbb{Q}(\beta) : \mathbb{Q}]$$

for its *absolute* algebraic degree. The two optimization problems are

$$\delta_+(\alpha) = \min \left\{ \max_{1 \leq i \leq k} \text{adeg}(\beta_i) : \alpha = \beta_1 + \cdots + \beta_k, k \geq 1 \right\}, \quad (1)$$

$$\delta_\times(\alpha) = \min \left\{ \max_{1 \leq i \leq k} \text{adeg}(\beta_i) : \alpha = \beta_1 \cdots \beta_k, k \geq 1 \right\}. \quad (2)$$

All terms lie in $\overline{\mathbb{Q}}$, with no requirement that they belong to $\mathbb{Q}(\alpha)$ or even its normal closure. Rational numbers have degree one. Zero has the trivial product representation 0, so $\delta_\times(0) = 1$.

These definitions optimize the *maximum degree*, not the number of terms, the coefficient heights of their minimal polynomials, radical nesting depth, or the printed length of the answer. A representation with more terms can be better under this objective. A rational coefficient multiplying a summand may always be absorbed into that summand; for nonzero $c \in \mathbb{Q}$,

$$\mathbb{Q}(c\beta) = \mathbb{Q}(\beta), \quad \text{adeg}(c\beta) = \text{adeg}(\beta). \quad (3)$$

Thus returning a rational linear combination of low-degree roots is an exact solution to the sum problem, not a relaxation of it.

The original discussion contains useful resultant and companion-matrix approaches, notably answers by Daniel Lichtblau and yarchik [1]. Here we develop their algebraic setting, distinguish successful searches from minimality proofs, and give a separate complete algorithm for the additive objective.

What is established here. Both examples from the question have globally optimal maximum degree 3. The sum problem has a finite, complete algorithm for every exact algebraic input, subject in the implementation to explicit resource limits. Product decomposition into two *specified* fields is exactly decidable by linear algebra. The general product routines return verified decompositions and report global optimality only when a separate lower bound proves it; unsuccessful bounded searches make no global negative claim.

1.1 Why a polynomial Root is not itself the degree certificate

The degree of the polynomial displayed inside a Root need not be the degree of the selected algebraic number if that polynomial is reducible. Nor does a root of a polynomial with algebraic coefficients necessarily have small absolute degree. For example, a quadratic equation over a large number field is not an absolute-degree-two certificate.

Accordingly, normalize the *value* and compute its minimal polynomial over $\mathbb{Q}$. RootReduce converts exact algebraic expressions to a standard root representation, while MinimalPolynomial computes the relevant polynomial [3, 4]. The package uses the following public interface:

```
Get["RootDecomposition.wl"];
AlgebraicDegree[alpha]
DegreeLowerBound[alpha]
```

Machine-real inputs are rejected. Increasing the precision of a machine number does not recover an exact algebraic value. All algebraic equalities below are checked symbolically, not by a tolerance on numerical approximations.

2 The two examples: discovery, reconstruction, and optimality

Put

$$p(x) = x^3 + x + 1, \quad q(x) = x^3 - x + 1, \quad a = \text{Root}(p, 1), \quad b = \text{Root}(q, 1). \quad (4)$$

Each cubic has exactly one real root. Their discriminants are -31 and -23 , respectively, and

$$a \approx -0.682327803828019, \quad b \approx -1.32471795724475.$$

The two polynomials in the question are

$$P_{\times}(x) = x^9 + 2x^7 - 3x^6 + x^5 - x^4 + 3x^3 - x - 1, \quad (5)$$

$$P_{+}(x) = x^9 + 6x^6 + 3x^5 - 15x^3 + 24x^2 - 4x + 8. \quad (6)$$

Both are irreducible over $\mathbb{Q}$ and have exactly one real root. The exact calculations below prove

$$\text{Root}(P_{\times}, 1) = ab, \quad \text{Root}(P_{+}, 1) = a + b. \quad (7)$$

The corresponding approximations are 0.903891894458348 and -2.00704576107277 . These decimal values are illustrations; the certificates do not depend on them.

2.1 Immediate Wolfram Language use

```
a = Root[1 + # + #^3 &, 1];
b = Root[1 - # + #^3 &, 1];
productRoot = Root[
  -1 - # + 3 #^3 - #^4 + #^5 - 3 #^6 + 2 #^7 + #^9 &, 1];
sumRoot = Root[
  8 - 4 # + 24 #^2 - 15 #^3 + 3 #^5 + 6 #^6 + #^9 &, 1];

productCertificate = SelectRootPair[
  productRoot, x^3 + x + 1, x^3 - x + 1, x, "Product"];
sumCertificate = SelectRootPair[
  sumRoot, x^3 + x + 1, x^3 - x + 1, x, "Sum"];
```

For each certificate, the mathematical result is

```
"Verified"      -> True
"Degrees"       -> {3, 3}
"MaximumDegree" -> 3
"InputDegree"   -> 9
"LowerBound"    -> 3
"Status"        -> "GloballyOptimal"
```

These are the intended package results, not a transcript of a native-kernel run. The "Terms" field contains the two exact algebraic values. The "Expression" field uses `Inactive[Plus]` or `Inactive[Times]` so that later simplification does not immediately combine the displayed decomposition back into one root.

2.2 Recovering the cubics rather than supplying them

For the sum, search the depressed-cubic template

$$p_0(t) = t^3 + At + B, \quad q_0(t) = t^3 + Ct + D.$$

The associated coefficient equations have a lexicographic Gröbner basis

$$A + C, \quad B - 1, \quad C^2 - 1, \quad D - 1. \quad (8)$$

The two solutions exchange the two cubics. In the package:

```
SolveRootTemplate[sumRoot,
  x^3 + u x + v, x^3 + w x + z,
  {u, v, w, z}, x, "Sum"]["Candidates"]
```

For the product, the restricted template

$$p_0(t) = t^3 + At + B, \quad q_0(t) = t^3 + Ct + 1$$

yields

$$A + C^5, \quad B - 1, \quad C^6 - 1. \quad (9)$$

Only $C = 1$ and $C = -1$ are rational, again recovering the desired cubics:

```
SolveRootTemplate[productRoot,
  x^3 + u x + v, x^3 + w x + 1,
  {u, v, w}, x, "Product"]["Candidates"]
```

Restricting the product template's constant coefficient to 1 is an *ansatz*, not a general normalization theorem. Its successful answer is exact, and its optimality follows independently from Section 4. A failure of this restricted template would not rule out another cubic product decomposition.

2.3 Transparent certificates inside the degree-nine fields

There is a useful exact check that does not require selecting two independent lists of roots. In the quotient field $\mathbb{Q}[x]/(P_\times)$ define

$$A_\times(x) = -\frac{x^7}{2} - x^5 + x^4 - \frac{1}{2}, \quad (10)$$

$$B_\times(x) = -\frac{x^7}{2} - x^5 + x^4 - x^3 - x + \frac{1}{2}. \quad (11)$$

Direct polynomial reduction gives

$$A_\times^3 + A_\times + 1 \equiv 0, \quad B_\times^3 - B_\times + 1 \equiv 0, \quad A_\times B_\times \equiv x \pmod{P_\times}. \quad (12)$$

At the real root of $P_\times$, both reconstructed values are real, so they must be the unique real roots a, b of the cubics.

For the sum, let

$$A_+(x) = \frac{3x^5 - 5x^3 - 3x^2 + 2x - 4}{6x^4 - 6x + 4}, \quad B_+(x) = x - A_+(x). \quad (13)$$

The denominator is relatively prime to P_+ . Inverting it modulo P_+ verifies

$$A_+^3 + A_+ + 1 \equiv 0, \quad B_+^3 - B_+ + 1 \equiv 0 \pmod{P_+}.$$

Again, reality identifies the intended branches. These identities are independently checked in `verify.py`. They also show explicitly that these particular decompositions happen to lie inside the input fields. That favorable feature must not be assumed for general inputs.

3 Resultants: exact pair detection

Suppose $p, q \in \mathbb{Q}[x]$ are monic of degrees r, s , with roots $u_1, \dots, u_r$ and $v_1, \dots, v_s$, counted with multiplicity. Define

$$R_+(x) = \text{Res}_t(p(t), q(x-t)), \quad (14)$$

$$R_\times(x) = \text{Res}_t(p(t), t^s q(x/t)). \quad (15)$$

The second argument in (15) is a polynomial. Implement it as

$$t^s q(x/t) = \sum_{j=0}^s q_j x^j t^{s-j}, \quad q(x) = \sum_{j=0}^s q_j x^j, \quad (16)$$

rather than relying on cancellation of a rational expression at $t = 0$.

The defining product formula for the resultant gives

$$R_+(x) = \prod_{i,j} (x - u_i - v_j), \quad R_\times(x) = \prod_{i,j} (x - u_i v_j). \quad (17)$$

Both are monic of degree rs . The documented `Resultant` operation supplies exactly this elimination primitive [5]. In the package,

```
ComposedRootPolynomial[p, q, x, "Sum"]
ComposedRootPolynomial[p, q, x, "Product"]
```

implements (14)–(16).

Proposition 3.1 (Exact pair criterion). *Let $f \in \mathbb{Q}[x]$ be the monic minimal polynomial of α . There is a pair of roots u of p and v of q with $\alpha = u + v$, respectively $\alpha = uv$, if and only if f divides R_+ , respectively $R_\times$.*

Proof. A matching pair makes α a root of the corresponding resultant, so irreducibility of f gives divisibility. Conversely, divisibility makes the specified α a zero of the product in (17). One of its linear factors must vanish. This implication is about the actual embedded root α , not merely an unspecified conjugate. $\square$

Repeated factors are removed before constructing the root lists, so `PolynomialRootList` enumerates distinct roots of the square-free polynomial. The returned polynomial and root-index metadata refer to that square-free version.

After the divisibility test, `SelectRootPair` enumerates the conjugates of the two candidate polynomials and accepts a pair only when

```
RootReduce[alpha - (beta + gamma)] == 0
(* or *)
RootReduce[alpha - beta gamma] == 0
```

is true. In a performance-oriented version, approximate values may order the candidate pairs, but they must not replace the exact final check.

3.1 Divisibility, not equality

A common shortcut matches the entire composed polynomial to the minimal polynomial of α . That is justified when both are monic and $rs = \text{adeg}(\alpha)$. In general, the correct constraint is

$$\text{rem}_x(R_\star, f) = 0, \quad \star \in \{+, \times\}. \quad (18)$$

For example,

$$\begin{aligned} R_+(x; x^2 - 2, x^2 - 8) &= (x^2 - 18)(x^2 - 2), \\ R_\times(x; x^2 - 2, x^2 - 3) &= (x^2 - 6)^2. \end{aligned}$$

The first reflects shared fields and different sums. The second reflects collisions among products, *even though* $\mathbb{Q}(\sqrt{2})$ and $\mathbb{Q}(\sqrt{3})$ are linearly disjoint. Linear disjointness alone does not make a sum or product a primitive element of the compositum.

More subtly, $\text{adeg}(\alpha)$ need not divide rs . Take the sum of a root of $t^3 - 2$ and a root of $t^3 + 2$:

$$R_+(x; t^3 - 2, t^3 + 2) = x^3(x^6 + 108). \quad (19)$$

The sextic factor is irreducible. Thus a difference of suitable cube-root conjugates has degree six although both terms have degree three. The universally valid easy inequality is $\text{adeg}(\alpha) \leq rs$, not $\text{adeg}(\alpha) \mid rs$. Enumerating only factor pairs of the input degree misses valid decompositions.

3.2 Companion matrices are the same construction

If C_p, C_q are companion matrices, then

$$\det(xI - C_p \otimes I - I \otimes C_q) = R_+(x), \quad \det(xI - C_p \otimes C_q) = R_\times(x).$$

This follows by passing to a splitting field and using the eigenvalues, with multiplicities. Resultants avoid constructing an rs by rs symbolic matrix. Companion matrices remain useful when tensor structure, sparse linear algebra, or characteristic-polynomial algorithms are advantageous. Neither formulation by itself solves the global optimization over all possible factor polynomials.

3.3 Coefficient matching and rationality

For a fixed polynomial template, expand the remainder in (18), set all its coefficients to zero, and solve for the template parameters. The package uses `Solve` as a candidate generator and then validates each specialization.

Only fully instantiated polynomials with *rational* coefficients are accepted. A complex solution for their coefficients does not certify the proposed absolute degrees. Nor is an unresolved family of parameter solutions evidence that no rational specialization exists. In particular, solving arbitrary polynomial equations over $\mathbb{Q}$ is not supplied here as a general-purpose decision oracle.

For reference, the depressed-cubic product formula is

$$\begin{aligned} R_\times(x) &= x^9 - 2ACx^7 - 3BDx^6 + A^2C^2x^5 + ABCDx^4 \\ &\quad + (A^3D^2 + B^2C^3 + 3B^2D^2)x^3 + AB^2CD^2x - B^3D^3. \end{aligned} \quad (20)$$

Equating this with $P_\times$ after $D = 1$ gives (9).

For sums, rational translation $(u, v) \mapsto (u + c, v - c)$ can remove one quadratic coefficient from a pair of monic cubics. In a full-degree-nine decomposition of the trace-zero example, the trace condition then removes the other one too. Product rescaling $(u, v) \mapsto (cu, c^{-1}v)$ is also valid for $c \in \mathbb{Q}^\times$, but it does not allow an arbitrary constant coefficient to be normalized to 1: the required rational power need not exist.

4 Global lower bounds

A successful search supplies an upper bound. A minimum needs a lower bound that applies to arbitrary numbers of terms and to possible terms outside the target field. The following elementary Galois argument gives such a bound. Standard facts on splitting fields, restriction maps, and the Galois correspondence are reviewed in Milne's notes [11].

Theorem 4.1 (Largest-prime-divisor bound). *Let $n = \text{adeg}(\alpha) > 1$, and let $P^+(n)$ be the largest prime divisor of n . Then*

$$\delta_+(\alpha) \geq P^+(n), \quad \delta_\times(\alpha) \geq P^+(n). \quad (21)$$

Proof. Suppose α is a sum or product of numbers β_i of degrees $m_i \leq d$. Let M_i be the normal closure of $\mathbb{Q}(\beta_i)$ and let M be their compositum. The Galois group of $M_i/\mathbb{Q}$ acts faithfully on the m_i conjugates of β_i , so it is a subgroup of S_{m_i} . Restriction embeds $\text{Gal}(M/\mathbb{Q})$ in the product of these finite groups. Every prime divisor of $[M : \mathbb{Q}]$ is therefore at most d . Since $\alpha \in M$, the degree n divides $[M : \mathbb{Q}]$. Hence $P^+(n) \leq d$. $\square$

Corollary 4.2. *Every algebraic number of prime degree p satisfies $\delta_+(\alpha) = \delta_\times(\alpha) = p$. For the two requested decompositions in (7), the optimum is exactly three.*

The degree-nine conclusion is stronger than saying that two quadratic terms are insufficient: *no finite number* of degree-one or degree-two terms can work.

Proposition 4.3 (Exponent obstruction). *Let L be the normal closure of $\mathbb{Q}(\alpha)$, and $G = \text{Gal}(L/\mathbb{Q})$. If either objective has a representation of maximum degree at most d , then*

$$\exp(G) \mid \text{lcm}(1, 2, \dots, d). \quad (22)$$

In particular, d is at least the largest prime power dividing $\exp(G)$. Also, if $d \leq 4$, then G must be solvable.

Proof. With M as above, its Galois group embeds in a product of subgroups of symmetric groups on at most d letters. Every element order divides $\text{lcm}(1, \dots, d)$. The group G is a quotient of $\text{Gal}(M/\mathbb{Q})$, since $L \subseteq M$. Exponents pass to subgroups and quotients. Symmetric groups on at most four letters are solvable, and solvability is preserved by products, subgroups, and quotients. $\square$

These are necessary conditions, not sufficient conditions. The additive degree-six example in Section 7 shows that representation-theoretic information can be stronger. The package's automatic generic certificate uses only (21); it does not pretend to compute a Galois group or its exponent.

5 Two specified number fields: both operations are linear

Suppose the candidate fields are known: $E = \mathbb{Q}(e)$ and $F = \mathbb{Q}(f)$, embedded together with α in some number field N . One can decide exactly whether

$$\alpha \in E + F \quad \text{or} \quad \alpha \in E^\times F^\times$$

using rational linear algebra. No assumption of linear disjointness is needed.

5.1 The additive test

Choose rational bases $e_1, \dots, e_r$ of E and $f_1, \dots, f_s$ of F . Write all values in a common rational basis of N , and solve

$$[\text{coord}(e_1) \cdots \text{coord}(e_r) \text{ coord}(f_1) \cdots \text{coord}(f_s)] \begin{pmatrix} u \\ v \end{pmatrix} = \text{coord}(\alpha). \quad (23)$$

Any solution returns $\beta = \sum u_i e_i \in E$ and $\gamma = \sum v_j f_j \in F$. Inconsistency proves that this specified field pair does not work.

5.2 The product test: a nonzero intersection

Theorem 5.1 (Fixed-field product criterion). *For $\alpha \neq 0$,*

$$\alpha \in E^\times F^\times \iff E \cap \alpha F \neq \{0\}. \quad (24)$$

Here $\alpha F = \{\alpha\eta : \eta \in F\}$ is a rational vector space, not generally a field.

Proof. If $\alpha = \beta\gamma$ with nonzero $\beta \in E$, $\gamma \in F$, then $\beta = \alpha\gamma^{-1} \in E \cap \alpha F$. Conversely, a nonzero $\beta = \alpha\eta$ in that intersection has $\eta \in F^\times$, so $\alpha = \beta\eta^{-1}$ is a suitable product. $\square$

Thus take the nullspace of

$$[\text{coord}(e_1) \cdots \text{coord}(e_r) - \text{coord}(\alpha f_1) \cdots - \text{coord}(\alpha f_s)]. \quad (25)$$

Any nonzero nullvector (u, v) gives

$$\beta = \sum_i u_i e_i, \quad \gamma = \left(\sum_j v_j f_j \right)^{-1}.$$

Neither expression in the denominator nor the E component can vanish: the two basis lists are independently linearly independent and $\alpha \neq 0$. This avoids a nonlinear bilinear coefficient system and avoids ideal factorization and unit computations entirely for this fixed-field question.

```
SplitOverFields[(1 + Sqrt[2]) (2 + Sqrt[3]),
  Sqrt[2], Sqrt[3], "Product"]

SplitOverFields[1 + Sqrt[2] + Sqrt[3],
  Sqrt[2], Sqrt[3], "Sum"]
```

The first has a product split and no sum split in these two fields; the second has a sum split and no product split. This can be checked in the basis $1, \sqrt{2}, \sqrt{3}, \sqrt{6}$: the first expression has nonzero mixed coefficient, while the second's product-coordinate matrix has rank two.

5.3 Getting consistent exact coordinates

The package's private `commonCoordinates` routine first calls

```
ToNumberField[values, All]
```

to put the finite input list in a common number field, extracts its primitive element θ , and re-embeds every value into that same field. The documented `All` setting requests the smallest common extension; the automatic setting need not do so [6].

For an algebraic integer generator θ , an `AlgebraicNumber[theta, list]` stores rational power-basis coefficients beginning with the constant coefficient [7]. The routine pads these lists with trailing zeros to a common length, handles rational values separately, and checks that the generators and coefficients agree before using the resulting matrix. These checks matter: comparing coordinate vectors from different primitive-element bases is not meaningful.

Exact consistency is tested by matrix rank, followed by `LinearSolve`; homogeneous product relations use `NullSpace`. The official linear-algebra interface accepts exact systems [8]. The cost of constructing the common field can dominate the later rational linear algebra.

6 A complete finite algorithm for sums

The additive problem has a finite search space depending only on the conjugates of the target. This is substantially stronger than searching polynomials of bounded coefficient height.

6.1 A preliminary descent to the normal closure

Let L be the normal closure of $\mathbb{Q}(\alpha)$. Suppose $\alpha = \sum_i \beta_i$, with the terms initially in an arbitrary extension. Choose a finite Galois extension $M/\mathbb{Q}$ containing L and all the terms, put $H = \text{Gal}(M/L)$, and define

$$T(z) = \frac{1}{|H|} \sum_{h \in H} h(z). \quad (26)$$

Then $T\alpha = \alpha$, each $T\beta_i$ belongs to L , and their sum is α . Since H is normal in $\text{Gal}(M/\mathbb{Q})$, T is equivariant under that group. Consequently, the stabilizer of β_i fixes $T\beta_i$, so

$$\text{adeg}(T\beta_i) \leq \text{adeg}(\beta_i).$$

Zero summands may be discarded. Therefore restricting the *sum* problem to the normal closure loses no possibilities.

Altamirano's 2017 manuscript proves an additive descent of this kind and formulates a subfield-based linear-algebra algorithm [12]. We now derive a different finite set of generators that avoids explicitly enumerating those subfields.

6.2 The conjugate-subset theorem

Let $\alpha_1, \dots, \alpha_n$ be all the distinct conjugates of α , in a fixed algebraic closure. For $S \subseteq \{1, \dots, n\}$ define

$$s_S = \sum_{i \in S} \alpha_i.$$

For $d \geq 1$, let

$$W_d = \text{span}_{\mathbb{Q}}\{s_S : \text{adeg}(s_S) \leq d\}. \quad (27)$$

Theorem 6.1 (Complete subset-sum criterion). *For every $\alpha \in \overline{\mathbb{Q}}$ and every integer $d \geq 1$,*

$$\delta_+(\alpha) \leq d \iff \alpha \in W_d. \quad (28)$$

In particular, arbitrary low-degree summands can be replaced, for this optimization objective, by rational multiples of low-degree subset sums of the conjugates of α .

Proof. The reverse implication is immediate from (3): a rational linear combination of such subset sums is a permitted decomposition.

For the forward implication, assume $\alpha = \sum_j \beta_j$ and $\text{adeg}(\beta_j) \leq d$. Choose a finite Galois extension $M/\mathbb{Q}$ containing all these numbers and all conjugates of α . Write $G = \text{Gal}(M/\mathbb{Q})$ and

$$V = \text{span}_{\mathbb{Q}}\{\alpha_1, \dots, \alpha_n\} \subset M.$$

This is a G -stable rational vector subspace. Start with any rational linear projection $P_0 : M \rightarrow V$ that is the identity on V , and average it:

$$P = \frac{1}{|G|} \sum_{g \in G} g P_0 g^{-1}. \quad (29)$$

The map P is a G -equivariant rational projection onto V , and $P\alpha = \alpha$.

Fix j and let H_j be the stabilizer of β_j in G . Then $P\beta_j \in V^{H_j}$ and

$$[G : H_j] = \text{adeg}(\beta_j) \leq d.$$

Consider the surjective G -equivariant map

$$\pi : \mathbb{Q}^n \longrightarrow V, \quad \pi(e_i) = \alpha_i,$$

where G permutes the standard basis according to its action on the conjugates. The induced map on H_j -fixed vectors is still surjective: lift a fixed vector arbitrarily and average the lift over H_j . The fixed subspace of this permutation module is spanned by the indicator vectors of the H_j -orbits on $\{1, \dots, n\}$. It follows that V^{H_j} is spanned by the corresponding orbit sums s_S .

Each such orbit sum is fixed by H_j , hence has degree at most $[G : H_j] \leq d$. Thus $P\beta_j \in W_d$ for every j . Finally,

$$\alpha = P\alpha = \sum_j P\beta_j \in W_d.$$

This argument accommodates any finite number of original summands and does not assume that their fields are subfields of the target field. $\square$

The proof uses only averaging over a finite group in characteristic zero. It neither assumes linear independence of the conjugates nor needs an explicit description of G for the resulting algorithm.

6.3 Halving the subset search and bounding the number of terms

The sum of all conjugates is rational:

$$\tau = \sum_{i=1}^n \alpha_i \in \mathbb{Q}.$$

Complementary subsets satisfy $s_{S^c} = \tau - s_S$, and rational translation or negation preserves the degree of a nonrational algebraic number. Therefore, after adjoining 1 to the spanning set, it suffices to use subsets of any chosen $n - 1$ conjugates. The raw candidate set contains at most

$$1 + (2^{n-1} - 1) = 2^{n-1}$$

values before zero removal and deduplication.

Moreover,

$$\dim_{\mathbb{Q}}(\mathbb{Q} + V) \leq n. \quad (30)$$

If $\tau \neq 0$, then $1 \in V$; if $\tau = 0$, the n conjugates satisfy a nontrivial rational relation. Hence a successful spanning computation needs at most n nonzero summands. This is a bound on a degree-optimal representation, not a claim that the routine minimizes its term count.

6.4 Algorithm and stopping certificate

The implementation proceeds as follows. Compute the minimal polynomial and all conjugates, construct their splitting field using a common primitive element θ , and store the roots as rational coordinate rows. Generate subset sums by adding those rows; do not repeatedly rebuild the common field. Compute each candidate's absolute degree, discard zero and duplicate candidates, and sort the remainder by degree.

Starting with an empty rational basis, retain a candidate only if it increases the span. After each addition, test whether the target coordinate row belongs to that span. On the first success, solve for the rational coefficients and return the corresponding algebraic terms. Since all lower-degree candidates have already been considered, Theorem 6.1 proves global minimality.

Candidates of degree at least n can be omitted when seeking improvement: the one-term answer α already has maximum degree n . If no lower-degree span contains the target, the same theorem proves that the trivial answer is optimal.

```
MinimumSumDecomposition[sumRoot]
MinimumSumDecomposition[productRoot]
MinimumSumDecomposition[Sqrt[2] + Sqrt[3] + Sqrt[5]]
```

The respective minimum maximum degrees are 3, 6, and 2. The last input has degree eight, but three quadratic summands suffice. Unlike a bounded-height search, this method can certify that a smaller maximum degree is impossible.

6.5 Complexity and explicit resource limits

The theorem is finite, not polynomial-time. There are exponentially many subsets, and the splitting field of a degree- n polynomial can have degree as large as $n!$. Algebraic-number construction and repeated minimal-polynomial calculations can be expensive even when the final rational span has dimension at most n .

The package defaults to

```
"MaxConjugates" -> 10
"MaxFieldDegree" -> 4096
```

These are resource guards, not mathematical restrictions. The field-degree check occurs *after* the common field has been constructed, so it cannot prevent that construction itself from taking substantial time. A wall-clock wrapper can be used:

```
TimeConstrained[
  MinimumSumDecomposition[alpha,
    "MaxConjugates" -> 12, "MaxFieldDegree" -> 10000],
  120, Failure["Timeout", <|>]]
```

A timeout or resource-guard failure is not a negative decomposition result. For larger problems, an explicit Galois action, subfield algorithms, or specialized representation-theoretic information can avoid constructing all candidates. Modern number-field subfield algorithms provide relevant alternatives [13]; no claim is made that the subset implementation is the fastest available method.

7 Changing the objective changes the answer

The product input ab has $\delta_{\times}(ab) = 3$, but its additive optimum is larger:

$$\boxed{\delta_{+}(ab) = 6.} \quad (31)$$

This supplies a concrete test of the complete additive algorithm beyond the two requested decompositions.

7.1 An explicit sum of two sextic roots

Let $a_1 = a, a_2, a_3$ be the roots of p and $b_1 = b, b_2, b_3$ the roots of q . Their sums vanish. For each permutation $\pi \in S_3$, define a matching sum

$$T_{\pi} = \sum_{i=1}^3 a_i b_{\pi(i)}.$$

Take the two permutations fixing 1. Their matching sums satisfy

$$\begin{aligned} T_1 + T_2 &= 2a_1 b_1 + (a_2 + a_3)(b_2 + b_3) \\ &= 3a_1 b_1. \end{aligned}$$

The common polynomial of the six matching sums is

$$R(x) = x^6 + 6x^4 - 27x^3 + 9x^2 - 81x + 4. \quad (32)$$

It is irreducible and has precisely two real roots. To obtain it directly, the two matching sums fixing (a_1, b_1) satisfy

$$T^2 - 3abT + 3a^2 - 3b^2 + 4 = 0. \quad (33)$$

Eliminating a, b using their two cubics produces $R(T)^3$.

Because a_2, a_3 and b_2, b_3 are conjugate pairs, both of the matching sums fixing 1 are real. They are distinct, their difference being $(a_2 - a_3)(b_2 - b_3)$ up to sign. They are therefore exactly the two real roots of R . This identifies the branches without relying on approximate coincidence:

$$ab = \frac{1}{3} \text{Root}(R, 1) + \frac{1}{3} \text{Root}(R, 2). \quad (34)$$

The two sextic roots are approximately 0.0496159784262872 and 2.66205970494876. Wolfram's polynomial-root ordering places real roots first in increasing order [2], so the indices in (34) are appropriate.

Each displayed rational coefficient can be absorbed into its root. An integer polynomial for $T/3$ is

$$R(3x) = 729x^6 + 486x^4 - 729x^3 + 81x^2 - 243x + 4.$$

Thus (34) is literally a sum of two degree-six polynomial roots if that output form is preferred.

7.2 Why degree five or less cannot suffice

The two cubic splitting fields have groups S_3 . Their unique quadratic subfields are $\mathbb{Q}(\sqrt{-31})$ and $\mathbb{Q}(\sqrt{-23})$, so their intersection is $\mathbb{Q}$. Indeed, a nontrivial common Galois quotient of two S_3 groups would force a common quadratic subfield. Hence their compositum L has group

$$G = S_3 \times S_3.$$

The rational span of the a_i is the two-dimensional standard representation U of S_3 , and similarly the span of the b_j is a standard representation W . Linear disjointness makes the multiplication map $U \otimes W \rightarrow L$ injective. Consequently the span of the nine conjugates $a_i b_j$ is

$$V \simeq U \otimes W,$$

a four-dimensional representation.

There are no nonzero vectors of V fixed by a subgroup of index below six. Such an index, if at most five, must divide 36, so it is 1, 2, 3, or 4. An index-two subgroup contains the commutator subgroup $A_3 \times A_3$, which has no fixed vectors on V . An index-four subgroup has order nine and is the unique Sylow-three subgroup $A_3 \times A_3$, with the same consequence.

For index three, the action on cosets gives a transitive homomorphism $G \rightarrow S_3$. Its image cannot be C_3 , because the abelianization of G is $C_2 \times C_2$. Thus the image is S_3 . The images of the two direct factors commute and are normal, so one is all of S_3 and the other is trivial. The subgroup therefore contains an entire S_3 factor, again giving no fixed vector in $U \otimes W$. Index one is immediate.

It follows that every nonzero subset sum of the nine conjugates has degree at least six. Theorem 6.1 rules out *any* additive representation of maximum degree below six, not merely a representation with two terms. Together with (34), this proves (31).

As an independent finite check, `verify.py` models the two standard representations with root vectors $(1, 0)$, $(0, 1)$, $(-1, -1)$, enumerates all 36 group actions and all subsets of eight conjugates, and recovers the same degree-six threshold by exact rational rank calculations.

8 Products can require factors outside the normal closure

For sums, normalized trace gives a degree-nonincreasing descent. A corresponding assertion for products is false. The following example also shows that the additive and multiplicative objectives can differ substantially.

Theorem 8.1 (A degree-eight obstruction). *Let*

$$\beta = \sqrt{1 + \sqrt{2}}, \quad \gamma = \sqrt{1 + \sqrt{7}}, \quad \alpha = \beta\gamma > 0. \quad (35)$$

Then

$$\text{adeg}(\alpha) = 8, \quad \delta_{\times}(\alpha) = 4, \quad \delta_{+}(\alpha) = 8. \quad (36)$$

Furthermore, in the normal closure L of $\mathbb{Q}(\alpha)$, no finite sum or product of elements of degree less than eight can equal α .

8.1 The polynomial and its splitting field

The two displayed factors have irreducible quartic polynomials

$$p_4(x) = x^4 - 2x^2 - 1, \quad q_4(x) = x^4 - 2x^2 - 6.$$

Their composed product is

$$R_{\times}(x; p_4, q_4) = F(x)^2, \quad F(x) = x^8 - 4x^6 - 40x^4 - 24x^2 + 36. \quad (37)$$

The polynomial F is irreducible. These exact polynomial statements are included in the independent symbolic checks. The quartic degrees also follow directly: $1 + \sqrt{2}$ and $1 + \sqrt{7}$ have negative rational norms, -1 and -6 , and therefore are not squares in their respective real quadratic fields.

The splitting fields of p_4 and q_4 are

$$L_{\beta} = \mathbb{Q}(\beta, i), \quad L_{\gamma} = \mathbb{Q}(\gamma, \sqrt{-6}),$$

each of degree eight and with dihedral group D_4 of order eight. For example, the roots of p_4 are $\pm\beta, \pm i/\beta$. Those of q_4 are $\pm\gamma, \pm\sqrt{-6}/\gamma$. The three quadratic subfields of the first splitting field have squareclasses

$$2, -1, -2;$$

those of the second have squareclasses

$$7, -6, -42.$$

There is no common quadratic field. Since a nontrivial common Galois quotient of these two 2-groups would itself have a quadratic quotient, the splitting fields are linearly disjoint. Their compositum M therefore has

$$\text{Gal}(M/\mathbb{Q}) = D_4 \times D_4, \quad [M : \mathbb{Q}] = 64.$$

In each D_4 , the central involution z changes the sign of all four roots. The kernel of the action on all products of a root of p_4 and a root of q_4 is precisely $\langle (z, z) \rangle$. To see that there are no additional kernel elements, an automorphism fixing every product must multiply every root of p_4 by the same scalar c , and every root of q_4 by c^{-1} . The sum of the squares of the roots of p_4 is 4, so $c^2 = 1$. The only possibilities are simultaneous identity or simultaneous sign change.

Thus the normal closure of $\mathbb{Q}(\alpha)$ is

$$L = M^{\langle (z, z) \rangle}, \quad G = \text{Gal}(L/\mathbb{Q}) = (D_4 \times D_4) / \langle (z, z) \rangle, \quad |G| = 32. \quad (38)$$

Its nontrivial central element c , represented by $(z, 1)$, sends α to $-\alpha$.

8.2 A subgroup obstruction to every smaller-degree internal factor

The center $Z = \{1, c\}$ of G has order two. The quotient G/Z is a four-dimensional vector space over $\mathbb{F}_2$. Commutation defines a nondegenerate alternating bilinear form on that quotient: each dihedral factor contributes a two-dimensional nondegenerate form, and the central product identifies their commutator values.

Let $H \leq G$ be a subgroup not containing c . Then $H \cap Z = \{1\}$, so H injects into G/Z . Since all commutators lie in Z , those of elements of H lie in $H \cap Z$ and vanish. The image of H is therefore a totally isotropic subspace of this four-dimensional nondegenerate alternating space. Its dimension is at most two, giving

$$c \notin H \implies |H| \leq 4 \implies [G : H] \geq 8. \quad (39)$$

Every element of L of absolute degree less than eight consequently has a stabilizer containing c and is fixed by c . Any sum or product of such elements is fixed by c , whereas α is not. This proves the last assertion of the theorem.

The additive descent of Section 6 would move any hypothetical external lower-degree additive representation into L without increasing the term degrees. Thus $\delta_+(\alpha) = 8$. The displayed quartic factors give $\delta_\times(\alpha) \leq 4$. Finally, G has exponent four, and 4 does not divide $\text{lcm}(1, 2, 3) = 6$. Proposition 4.3 gives $\delta_\times(\alpha) \geq 4$, completing the proof of (36).

The finite-group program independently enumerates all 110 subgroups of the group (38) and confirms that a subgroup avoiding c has order at most four. This check supplements, rather than replaces, the structural proof.

8.3 A relevant manuscript needs a correction

Altamirano's manuscript states, in Theorem 3.1, that a two-factor lower-degree product decomposition can always be moved into the normal closure of the target. Its proof sets a new factor to a_0/a_1 , where a_1 is a coefficient of a relative minimal polynomial, without establishing that $a_1 \neq 0$ [12]. Theorem 8.1 disproves the stated reduction, not merely this step of its proof. In this example, the relative polynomial of β over L is quadratic with zero linear coefficient: simultaneous sign change is the nontrivial automorphism of M/L .

The additive descent remains valid. The correction is specifically important for implementation: enumerating all subfields of the target's normal closure is a complete additive strategy, but an incomplete multiplicative one.

8.4 Why taking norms does not repair the reduction

Suppose $\alpha = \prod_i \beta_i$ in a finite Galois extension M containing L . Taking norms gives

$$\alpha^{[M:L]} = \prod_i N_{M/L}(\beta_i). \quad (40)$$

The factors on the right lie in L , and normality shows that their degrees do not exceed those of the original factors. But (40) represents a *power* of α , not α . Taking roots can increase degrees and introduce root-of-unity choices. Already $\alpha = \sqrt{2}$ has a rational square but is not a product of rational numbers. Membership after taking a power is therefore not a substitute for the original degree-constrained multiplicative membership test.

9 Certified product searches and a practical workflow

A useful implementation can combine several complementary methods without conflating their guarantees.

9.1 Finite rational-polynomial catalogs

For integers $d, H \geq 1$, enumerate primitive integer polynomials of degrees at most d with coefficients of absolute value at most H and positive leading coefficient. Keep irreducible polynomials and make them monic over $\mathbb{Q}$. This is a finite catalog, and letting H increase covers every rational minimal polynomial of degree at most d .

```
catalog = PolynomialCatalog[3, 1, x,
    "MonicOnly" -> True];
SearchRootCatalog[productRoot, catalog, x, "Product"]
```

The tiny monic height-one catalog includes the two desired cubics. The monic-only restriction is useful for algebraic-integer examples but is not exhaustive for arbitrary algebraic numbers: a nonintegral algebraic number need not have a monic *integer* polynomial. The default catalog allows nonunit leading coefficients. Irreducibility is checked by the documented `IrreduciblePolynomialQ` predicate [9].

The pair search orders candidates by maximum polynomial degree, rejects pairs whose degree product is below the input degree, tests resultant divisibility, and selects a certified exact conjugate pair. With irreducible catalog polynomials, its first successful pair minimizes the maximum degree *among that catalog's pairs*. With arbitrary user-supplied reducible polynomials, the final minimal-polynomial check remains sound, but that first-success ordering need not optimize even the catalog's actual root degrees.

9.2 More than two factors: exact residual search

For a fixed degree threshold d , height catalog, and factor-count limit k , choose tentative factors $\beta_1, \dots, \beta_j$ and compute the exact residual

$$\rho = \text{RootReduce}(\alpha / (\beta_1 \cdots \beta_j)). \quad (41)$$

If $\text{adeq}(\rho) \leq d$, append it as the last factor and verify the product. Otherwise continue until the factor-count bound is reached. The residual itself is not constrained by the catalog's coefficient height, which often makes this substantially more effective than choosing all factors independently.

```
SearchProductResidual[productRoot, 3,
    {x^3 + x + 1, x^3 - x + 1}, x, 2]
```

The routine removes zero and unit trial factors, allows repetitions, and traverses trial factors in nondecreasing catalog order to avoid permutation duplicates. It permits fewer than k factors by testing the residual at each recursion level.

9.3 What exhaustive enumeration does and does not establish

For a fixed d , dovetailing coefficient heights and factor-count limits eventually finds every existing finite product representation of maximum degree at most d . Indeed, all but the last factor of a particular representation occur in some finite catalog, and the last factor is then accepted by the residual test. Thus the procedure is complete as a *positive search*.

It is not, as supplied, a terminating negative decision procedure. If no such representation exists, increasing heights and term counts does not by itself certify that fact. Likewise, trying all degree pairs with product equal to the input degree is not a substitute: Section 3 explains why it is incomplete.

An anytime optimizer can retain the best certified decomposition found while dovetailing searches below its current maximum degree. Its best value eventually reaches the true minimum, because a finite witness for

that minimum exists. Without a matching lower bound, however, it need not know when that stage has been reached. The package deliberately distinguishes "CertifiedUpperBound" from "GloballyOptimal".

9.4 Recommended order of computation

Begin with the exact minimal polynomial and the largest-prime lower bound. Prime-degree inputs are finished immediately. For a sum, use a cheap template or catalog search first when the input suggests one; if its upper bound matches the lower bound, stop. Otherwise use the complete subset algorithm when its resource requirements are acceptable.

For a product, try low-degree templates, small coefficient-height catalogs, or a fixed-field intersection test when candidate fields are available. Add a residual search when more factors may help. Every discovered decomposition should pass the same exact verifier. Add Galois-theoretic lower bounds when available, but do not infer global nonexistence from a timeout, an unresolved coefficient system, an empty bounded catalog search, or a search restricted to the normal closure.

Method	Positive result	Negative conclusion
Resultant template	Exact verified candidate	Only within completely solved template; no global claim
Finite catalog pair search	Exact verified pair	No pair in that catalog
Fixed-field linear algebra	Exact split in E, F	No split in those specified fields
Complete conjugate-subset sum search	Globally optimal sum	No lower-degree sum, if search completes
Bounded product residual search	Exact verified product	No product within stated search bounds
Global lower bound plus matching answer	Globally optimal result	Smaller maximum degree impossible

10 Implementation details, validation, and reproducibility

10.1 Package interfaces and output contract

The complete source is included as `RootDecomposition.wl` and reproduced in Appendix A. The main user-facing functions are summarized below.

Function	Purpose
<code>AlgebraicDegree</code>	Absolute minimal-polynomial degree
<code>DegreeLowerBound</code>	Largest-prime global lower bound
<code>ComposedRootPolynomial</code>	Sum or product resultant
<code>PolynomialRootList</code>	Exact conjugate enumeration
<code>VerifyRootDecomposition</code>	Exact equality and degree certificate
<code>SelectRootPair</code>	Divisibility test and branch selection
<code>SolveRootTemplate</code>	Rational specializations of a coefficient template
<code>PolynomialCatalog</code>	Bounded-height irreducible rational polynomials
<code>SearchRootCatalog</code>	Finite pair search
<code>SearchProductResidual</code>	Bounded multi-factor search
<code>SplitOverFields</code>	Exact decision for a specified field pair
<code>MinimumSumDecomposition</code>	Complete additive optimization, subject to guards

Every successful certificate records the exact input, operation, terms, individual absolute degrees, their maximum, the input degree, an available lower bound, and the reason for any optimality assertion. Zero

summands and unit factors are removed where appropriate. The generic verifier only claims global optimality when its upper bound equals the prime lower bound. The complete additive algorithm may make a stronger assertion using Theorem 6.1.

Early exits from nested searches use uniquely tagged `Catch/Throw` pairs. This is deliberate: a `Return` inside a `Do` loop can exit only the loop rather than the surrounding function [10]. Allowing execution to fall through to a negative result, or to the additive algorithm’s trivial fallback, would invalidate the result.

The package is not a polished large-scale algebraic-number library. In particular, it does not implement common-field caching across independent calls, subgroup enumeration, height prediction, sparse incremental elimination, or adaptive numerical root ordering. Such improvements can affect practical speed without changing the exact acceptance checks.

10.2 Executed independent checks

The supplied `verify.py` was executed with SymPy 1.14.0. All 50 mathematical assertions passed. The machine-readable results are in `verification_results.json`. The assertions cover exact resultants, irreducibility, Sturm root counts and isolating intervals, polynomial reconstruction identities, both Gröbner bases, repeated-root and degree-drop examples, the sextic matching construction, and the quartic-product counterexample.

They also cover exact finite-group models of the subset algorithm. For the nine sum conjugates and the nine product conjugates, deduplication leaves 133 nonzero subset-sum vectors in each model. Their degree histograms differ:

Degree	Conjugates of $a + b$	Conjugates of ab
3	6	0
6	12	6
9	55	27
12	0	12
18	36	52
36	24	36

These counts exclude the extra rational candidate 1, and refer to the script’s specified choice of the omitted conjugate and duplicate representatives. They are not complexity estimates for arbitrary inputs. Exact span tests recover optima 3 and 6. The multiquadratic degree-eight example similarly yields optimum 2.

For the order-32 counterexample group, the subgroup-size counts are

Subgroup order	1	2	4	8	16	32
Count	1	19	39	35	15	1

All subgroups of index below eight contain the central involution that negates α . Finally, exact four-dimensional coordinate tests verify both positive and negative fixed-field sum and product cases.

10.3 Native Wolfram execution status

The available Wolfram evaluator endpoint returned HTTP 404, so the accompanying Wolfram Language package and its native regression tests were *not executed in a native Wolfram kernel* during preparation. Independent symbolic tests support the mathematics and the algorithms, but they do not establish that every

Wolfram Language evaluation detail has been exercised. No kernel version, performance timing, or native test-pass count is claimed.

On a local kernel, run the supplied files with

```
Get["examples.wl"]
TestReport["tests.wlt"]
TestReport["slow-tests.wlt"]
```

The ordinary test file contains 28 tests, including input rejection, exact branch selection, template recovery, residual search, and fixed-field decisions. The separate four-test file exercises the potentially expensive complete additive searches. The slower tests are separated to keep routine regression checks manageable.

For independent verification and rebuilding the document:

```
python verify.py
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The PDF uses standard TeX packages and reads the accompanying WL source for its appendix.

11 Conclusion

The two original examples admit more than plausible-looking simplifications: exact resultants recover the cubics, explicit field identities certify the selected real roots, and a prime-divisor argument proves that maximum degree three is globally minimal.

For sums, the conjugate-subset theorem supplies a finite global optimization method. It searches a structured family derived from the target itself, not an unbounded collection of possible summand polynomials. For a fixed pair of fields, products are also simpler than they first appear: $E \cap \alpha F$ provides a complete rational linear-algebra test.

The unrestricted product problem must nevertheless be treated separately. Low-degree factors can genuinely lie outside the target's normal closure, as the degree-eight example proves. Exact searches remain useful and can certify optimality whenever matched by a valid lower bound. The essential implementation discipline is to preserve the distinction between a witness, a bounded negative result, and a proof of global minimality.

A Complete Wolfram Language package

The file below is the source shipped in the archive. Load it with `Get`; the `examples.wl`, `tests.wlt`, and `slow-tests.wlt` files provide usage and regression cases. Its native execution status is described in Section 10.

```

1 (* ::Package:: *)
2 (* Exact algebraic Root decomposition. See article.pdf for guarantees.
3    This source was reviewed but NOT executed in a native Wolfram kernel
4    during preparation: the available evaluator endpoint returned HTTP 404. *)
5
6 BeginPackage["RootDecomposition"];
7
8 AlgebraicDegree::usage = "AlgebraicDegree[a] gives the absolute degree of an exact algebraic number.";
9 DegreeLowerBound::usage = "DegreeLowerBound[a] gives a global lower bound for the maximum degree in any sum
   or product decomposition.";
10 ComposedRootPolynomial::usage = "ComposedRootPolynomial[p,q,x,op] constructs the monic composed polynomial;
   op is \"Sum\" or \"Product\".";
11 PolynomialRootList::usage = "PolynomialRootList[p,x] lists the distinct exact roots of a rational polynomial
   in square-free Root order.";
12 VerifyRootDecomposition::usage = "VerifyRootDecomposition[a,terms,op] verifies equality and reports degrees
   and any prime-bound optimality certificate.";
13 SelectRootPair::usage = "SelectRootPair[a,p,q,x,op] selects an exactly matching pair of conjugates from
   rational polynomials p and q.";
14 SolveRootTemplate::usage = "SolveRootTemplate[a,p,q,vars,x,op] solves a fixed polynomial template and accepts
   only fully instantiated rational-coefficient candidates. It is not a global negative decision procedure.
   ";
15 PolynomialCatalog::usage = "PolynomialCatalog[d,h,x] enumerates primitive integer polynomials of degree at
   most d and coefficient height at most h, retaining irreducible polynomials and returning monic rational
   versions.";
16 SearchRootCatalog::usage = "SearchRootCatalog[a,polys,x,op] searches pairs from a finite polynomial catalog.";
17 SearchProductResidual::usage = "SearchProductResidual[a,d,polys,x,k] searches products with at most k factors,
   choosing all but the last factor from polys and testing the exact residual degree.";
18 SplitOverFields::usage = "SplitOverFields[a,e,f,op] decides whether a is a sum/product of elements of Q(e)
   and Q(f), by exact rational linear algebra.";
19 MinimumSumDecomposition::usage = "MinimumSumDecomposition[a] computes a globally degree-minimal sum
   decomposition by the conjugate-subset theorem, subject to explicit resource guards.";
20
21 Begin["Private"];
22
23 rationalQ[z_] := MatchQ[z, _Integer | _Rational];
24 exactAlgebraicQ[z_] := FreeQ[z, _Real] && TrueQ[Element[z, Algebraics]];
25 zeroQ[z_] := TrueQ[Quiet[Check[RootReduce[z], $Failed]] === 0];
26 err[tag_, text_, extra_ <|>] := Failure[tag, Join[<|"MessageTemplate" -> text|>, extra]];
27 operationQ[op_] := MemberQ[{"Sum", "Product"}, op];
28 rationalPolynomialQ[p_, x_Symbol] := PolynomialQ[p, x] &&
29   Exponent[p, x] >= 1 && AllTrue[CoefficientList[Expand[p], x], rationalQ];
30 monic[p_, x_Symbol] := Expand[p/Coefficient[p, x, Exponent[p, x]]];
31 squareFreeMonic[p_, x_Symbol] := monic[Cancel[p/PolynomialGCD[p, D[p, x]], x]];
32
33 AlgebraicDegree[a_] := Module[{x = Unique["x$"], p},
34   If[!exactAlgebraicQ[a], Return[err["NotExactAlgebraic", "Expected an exact algebraic number."]]];
35   p = Quiet[Check[MinimalPolynomial[a, x], $Failed]];
36   If[p === $Failed || !rationalPolynomialQ[p, x],
37     err["MinimalPolynomialFailed", "Could not compute a rational minimal polynomial."],
38     Exponent[p, x]]
39 ];
40
41 DegreeLowerBound[a_] := Module[{n = AlgebraicDegree[a]},
42   If[FailureQ[n], Return[n]];
43   If[n == 1, 1, Max[First /@ FactorInteger[n]]]
44 ];
45
46 ComposedRootPolynomial[p_, q_, x_Symbol, op_] := Module[
47   {t = Unique["t$"], pp, qq, s, transformed, r},

```

```

48 If[!operationQ[op], Return[err["Operation", "Operation must be Sum or Product."]]];
49 If[!PolynomialQ[p, x] || !PolynomialQ[q, x] ||
50   Exponent[p, x] < 1 || Exponent[q, x] < 1,
51   Return[err["Polynomial", "Expected two nonconstant polynomials."]]];
52 pp = monic[p, x]; qq = monic[q, x]; s = Exponent[qq, x];
53 transformed = If[op === "Sum", qq /. x -> (x - t),
54   Sum[Coefficient[qq, x, j] x^j t^(s - j), {j, 0, s}]];
55 r = Resultant[pp /. x -> t, transformed, t];
56 Expand[r]
57 ];
58
59 PolynomialRootList[p_, x_Symbol] := Module[{q, fun, n},
60   If[!rationalPolynomialQ[p, x],
61     Return[err["RationalPolynomial", "Root polynomials must have rational coefficients."]]];
62   q = squareFreeMonic[p, x]; n = Exponent[q, x];
63   fun = Function[Evaluate[q /. x -> Slot[1]]];
64   Table[Root[fun, j], {j, 1, n}]
65 ];
66
67 VerifyRootDecomposition[a_, supplied_List, op_] := Module[
68   {terms, value, degrees, n, lb, d, optimal},
69   If[!operationQ[op], Return[err["Operation", "Operation must be Sum or Product."]]];
70   If[!exactAlgebraicQ[a] || supplied === {} || !AllTrue[supplied, exactAlgebraicQ],
71     Return[err["Input", "Expected exact algebraic input and a nonempty list of exact terms."]]];
72   terms = RootReduce /@ supplied;
73   value = If[op === "Sum", Total[terms], Times @@ terms];
74   If[!zeroQ[a - value], Return[err["NotEqual", "The proposed decomposition is not exactly equal to the input."]]];
75   terms = If[op === "Sum", Select[terms, !zeroQ[#] &], Select[terms, !zeroQ[# - 1] &]];
76   If[terms === {}, terms = {If[op === "Sum", 0, 1]}];
77   degrees = AlgebraicDegree /@ terms; n = AlgebraicDegree[a];
78   If[AnyTrue[degrees, FailureQ] || FailureQ[n], Return[err["DegreeFailed", "Degree computation failed."]]];
79   lb = DegreeLowerBound[a]; d = Max[degrees]; optimal = (d == lb);
80   <|"Input" -> a, "Operation" -> op, "Terms" -> terms,
81     "Expression" -> If[op === "Sum", Inactive[Plus] @@ terms, Inactive[Times] @@ terms],
82     "Verified" -> True, "Degrees" -> degrees, "MaximumDegree" -> d,
83     "InputDegree" -> n, "LowerBound" -> lb,
84     "Status" -> If[optimal, "GloballyOptimal", "CertifiedUpperBound"],
85     "OptimalityReason" -> If[optimal, "LargestPrimeDivisorBound", "NotEstablished"]>|>
86 ];
87
88 SelectRootPair[a_, p_, q_, x_Symbol, op_] := Module[
89   {f, r, pr, qr, v, cert, tag = Unique["pair$"]},
90   If[!operationQ[op] || !exactAlgebraicQ[a], Return[err["Input", "Invalid input or operation."]]];
91   If[!rationalPolynomialQ[p, x] || !rationalPolynomialQ[q, x],
92     Return[err["RationalPolynomial", "Candidate coefficients must all be rational."]]];
93   f = monic[MinimalPolynomial[a, x], x];
94   r = ComposedRootPolynomial[p, q, x, op];
95   If[Expand[PolynomialRemainder[r, f, x]] != 0,
96     Return[err["NoPair", "The minimal polynomial does not divide the composed polynomial."]]];
97   pr = PolynomialRootList[p, x]; qr = PolynomialRootList[q, x];
98   Catch[
99     Do[
100       v = If[op === "Sum", pr[[i]] + qr[[j]], pr[[i]] qr[[j]]];
101       If[zeroQ[a - v],
102         cert = VerifyRootDecomposition[a, {pr[[i]], qr[[j]]}, op];
103         If[!FailureQ[cert], Throw[Join[cert,
104           <|"Polynomials" -> {squareFreeMonic[p, x], squareFreeMonic[q, x]},
105             "RootIndices" -> {i, j}>|, tag]],
106         {i, Length[pr]}, {j, Length[qr]}];
107       err["RootSelectionFailed", "Divisibility held but no exact conjugate pair was selected."],
108       tag]
109 ];
110
111 SolveRootTemplate[a_, p_, q_, vars_List, x_Symbol, op_] := Module[
112   {f, r, remainder, equations, solutions, pp, qq, cert, out = {}, rejected = {}},
113   If[!exactAlgebraicQ[a] || !operationQ[op], Return[err["Input", "Invalid exact input or operation."]]];
114   f = monic[MinimalPolynomial[a, x], x];

```

```

115 r = ComposedRootPolynomial[p, q, x, op];
116 If[FailureQ[r], Return[r]];
117 remainder = Expand[PolynomialRemainder[r, f, x]];
118 equations = (# == 0 &) /@ CoefficientList[remainder, x];
119 solutions = Quiet[Check[Solve[equations, vars], $Failed]];
120 If[solutions === $Failed || !ListQ[solutions] || !AllTrue[solutions, ListQ],
121   Return[err["SolverUnresolved", "Solve did not return a usable list of rule lists.",
122     <|"Equations" -> equations|>]];
123 Do[
124   pp = Expand[p /. rules]; qq = Expand[q /. rules];
125   If[rationalPolynomialQ[pp, x] && rationalPolynomialQ[qq, x],
126     cert = SelectRootPair[a, pp, qq, x, op];
127     If[!FailureQ[cert], AppendTo[out, cert], rejected++, rejected++],
128   {rules, solutions}];
129 <|"Status" -> "TemplateSearchCompleted", "Candidates" -> DeleteDuplicates[out],
130   "Equations" -> equations, "SolverRules" -> solutions,
131   "RejectedOrUnresolvedCount" -> rejected,
132   "NegativeGlobalConclusion" -> False|>
133 ];
134
135 Options[PolynomialCatalog] = {"MonicOnly" -> False};
136 PolynomialCatalog[d_Integer?Positive, h_Integer?Positive, x_Symbol, OptionsPattern[]] := Module[
137   {out = {}, leading, v, p},
138   leading = If[TrueQ[OptionValue["MonicOnly"]], {1}, Range[h]];
139   Do[
140     Do[
141       v = Append[c, lc];
142       If[Apply[GCD, Abs[v]] == 1,
143         p = v . x^Range[0, m];
144         If[TrueQ[IrreduciblePolynomialQ[p]], AppendTo[out, monic[p, x]]],
145       {lc, leading}, {c, Tuples[Range[-h, h], m]}],
146     {m, 1, d}];
147   DeleteDuplicates[out]
148 ];
149
150 SearchRootCatalog[a_, polys_List, x_Symbol, op_] := Module[
151   {ps, n, pairs, cert, tag = Unique["catalog$"]},
152   If[!exactAlgebraicQ[a] || !operationQ[op], Return[err["Input", "Invalid input."]]];
153   If[!AllTrue[polys, rationalPolynomialQ[#, x] &], Return[err["Catalog", "Invalid polynomial catalog."]]];
154   ps = DeleteDuplicates[monic[#, x] & /@ polys]; n = AlgebraicDegree[a];
155   pairs = Flatten[Table[{i, j}, {i, Length[ps]}, {j, i, Length[ps]}, 1];
156   pairs = Select[pairs, Exponent[ps[[#[[1]]]], x] Exponent[ps[[#[[2]]]], x] >= n &];
157   pairs = SortBy[pairs, Max[Exponent[ps[[#[[1]]]], x], Exponent[ps[[#[[2]]]], x]] &];
158   Catch[
159     Do[
160       cert = SelectRootPair[a, ps[[ij[[1]]]], ps[[ij[[2]]]], x, op];
161       If[!FailureQ[cert], Throw[cert, tag]];
162       If[cert[[1]] != "NoPair", Throw[cert, tag], {ij, pairs}];
163     err["NotFoundInCatalog", "No matching pair was found in this finite catalog; this is not global
164       nonexistence."],
165     tag]
166 ];
167
168 SearchProductResidual[a_, d_Integer?Positive, polys_List, x_Symbol, k_Integer?Positive] := Module[
169   {roots, degrees, walk, answer, tag = Unique["found$"]},
170   If[!exactAlgebraicQ[a] || !AllTrue[polys, rationalPolynomialQ[#, x] &],
171     Return[err["Input", "Invalid exact input or catalog."]]];
172   roots = Quiet[Check[DeleteDuplicates[RootReduce /@
173     Flatten[PolynomialRootList[#, x] & /@ polys]], $Failed]];
174   If[roots === $Failed, Return[err["CatalogEvaluationFailed", "Could not evaluate the exact catalog roots."
175     ]]];
176   degrees = AlgebraicDegree /@ roots;
177   If[AnyTrue[degrees, FailureQ], Return[err["DegreeFailed", "A catalog root degree could not be computed."]]];
178
179   roots = Pick[roots, MapThread[TrueQ[#2 <= d] && !zeroQ[#1] && !zeroQ[#1 - 1] &, {roots, degrees}]];
180   walk[residual_, prefix_List, start_Integer] := Module[{degree, next, cert},
181     degree = AlgebraicDegree[residual];
182     If[FailureQ[degree], Throw[degree, tag]];

```



```

180   If[degree <= d,
181     cert = VerifyRootDecomposition[a, Append[prefix, residual], "Product"];
182     If[!FailureQ[cert], Throw[cert, tag]];
183   If[Length[prefix] >= k - 1, Return[Null]];
184   Do[next = RootReduce[residual/roots[[i]]];
185     walk[next, Append[prefix, roots[[i]]], i], {i, start, Length[roots]}}
186 ];
187 answer = Catch[walk[a, {}, 1]; Missing["NotFound"], tag];
188 If[MissingQ[answer], err["NotFoundWithinBounds", "No decomposition was found within the supplied catalog
    and factor-count bound."], answer]
189 ];
190
191 (* Consistent rational coordinates in one integral primitive-element basis. *)
192 commonCoordinates[values_List] := Module[{nf, sample, theta, n, vectors, z},
193   nf = Quiet[Check[ToNumberField[values, All], $Failed]];
194   If[nf === $Failed || !ListQ[nf], Return[err["NumberFieldFailed", "ToNumberField did not produce a common
    field."]]];
195   If[AllTrue[nf, rationalQ], Return[<|"Generator" -> 0, "Degree" -> 1, "Rows" -> (List /@ nf)|>]];
196   sample = SelectFirst[nf, MatchQ[#, _AlgebraicNumber] &], Missing["Generator"]];
197   If[MissingQ[sample], Return[err["GeneratorFailed", "No explicit common-field generator was returned."]]];
198   theta = sample[[1]]; n = AlgebraicDegree[theta];
199   If[FailureQ[n], Return[n]];
200   vectors = Table[
201     z = If[rationalQ[v], v, Quiet[Check[ToNumberField[v, theta], $Failed]]];
202     Which[rationalQ[z], PadRight[{z}, n],
203       MatchQ[z, _AlgebraicNumber] && SameQ[z[[1]], theta] &&
204       Length[z[[2]]] <= n && AllTrue[z[[2]], rationalQ], PadRight[z[[2]], n],
205       True, $Failed], {v, nf}];
206   If[MemberQ[vectors, $Failed], Return[err["CoordinatesFailed", "Could not establish a consistent rational
    power basis."]]];
207   <|"Generator" -> theta, "Degree" -> n, "Rows" -> vectors|>
208 ];
209
210 SplitOverFields[a_, e_, f_, op_] := Module[
211   {r, s, eb, fb, data, rows, target, mat, sol, null, beta, gamma, cert},
212   If[!operationQ[op] || !AllTrue[{a, e, f}, exactAlgebraicQ], Return[err["Input", "Expected exact algebraic
    inputs."]]];
213   If[op === "Product" && zeroQ[a], Return[VerifyRootDecomposition[a, {0, 1}, op]];
214   r = AlgebraicDegree[e]; s = AlgebraicDegree[f];
215   If[FailureQ[r] || FailureQ[s], Return[err["DegreeFailed", "Field-degree computation failed."]]];
216   eb = e^Range[0, r - 1]; fb = f^Range[0, s - 1];
217   If[op === "Sum",
218     data = commonCoordinates[Join[{a}, eb, fb]];
219     If[FailureQ[data], Return[data]];
220     rows = data["Rows"]; target = First[rows]; mat = Transpose[Rest[rows]];
221     If[MatrixRank[Rest[rows]] != MatrixRank[rows],
222       Return[err["NoSplitInFields", "The input is not in the sum of the specified fields."]]];
223     sol = Quiet[Check[LinearSolve[mat, target], $Failed]];
224     If[sol === $Failed, Return[err["LinearSolveFailed", "Exact linear solving failed."]]];
225     beta = Take[sol, r] . eb; gamma = Drop[sol, r] . fb,
226     data = commonCoordinates[Join[eb, a fb]];
227     If[FailureQ[data], Return[data]];
228     rows = data["Rows"];
229     mat = Transpose[Join[Take[rows, r], -Drop[rows, r]]];
230     null = NullSpace[mat];
231     If[null === {}, Return[err["NoSplitInFields", "The intersection E with a F is zero."]]];
232     sol = First[null]; beta = Take[sol, r] . eb;
233     gamma = 1/(Drop[sol, r] . fb)
234 ];
235   cert = VerifyRootDecomposition[a, {beta, gamma}, op];
236   If[FailureQ[cert], cert, Join[cert, <|"FieldGenerators" -> {e, f}, "Method" -> "RationalLinearAlgebra"|>]]
237 ];
238
239 Options[MinimumSumDecomposition] = {"MaxConjugates" -> 10, "MaxFieldDegree" -> 4096};
240 MinimumSumDecomposition[a_, OptionsPattern[]] := Module[
241   {n, lb, x = Unique["x$"], f, roots, data, theta, dimension, rootRows,
242   target, candidates, entries, basis = {}, row, proposed, rank, sol,
243   terms, cert, one, zero, candidateCount, tag = Unique["sum$"]},

```



```

244 n = AlgebraicDegree[a]; If[FailureQ[n], Return[n]];
245 lb = DegreeLowerBound[a];
246 If[n == lb, Return[VerifyRootDecomposition[a, {a}, "Sum"]]];
247 If[TrueQ[n > OptionValue["MaxConjugates"]],
248   Return[err["ConjugateBudget", "Increase MaxConjugates to permit the exponential subset search.", <|"
    InputDegree" -> n|>]]];
249 f = monic[MinimalPolynomial[a, x], x]; roots = PolynomialRootList[f, x];
250 data = commonCoordinates[Prepend[roots, a]];
251 If[FailureQ[data], Return[data]];
252 theta = data["Generator"]; dimension = data["Degree"];
253 If[TrueQ[dimension > OptionValue["MaxFieldDegree"]],
254   Return[err["FieldBudget", "The constructed splitting field exceeds MaxFieldDegree.", <|"FieldDegree" ->
    dimension|>]]];
255 target = First[data["Rows"]]; rootRows = Rest[data["Rows"]];
256 one = UnitVector[dimension, 1]; zero = ConstantArray[0, dimension];
257 candidates = DeleteDuplicates[Join[{one}, Total /@ Subsets[Most[rootRows], {1, n - 1}]]];
258 candidates = Select[candidates, !SameQ[#, zero] &]; candidateCount = Length[candidates];
259 entries = {AlgebraicDegree[AlgebraicNumber[theta, #]], #} & /@ candidates;
260 If[AnyTrue[entries, FailureQ[First[#]] &], Return[err["CandidateDegreeFailed", "A subset-sum degree could
    not be computed."]]];
261 (* Values of degree n or greater cannot improve on the trivial answer. *)
262 entries = SortBy[Select[entries, First[#] < n &], First];
263 Catch[
264   Do[
265     row = entry[[2]]; proposed = Append[basis, row]; rank = MatrixRank[proposed];
266     If[rank > Length[basis],
267       basis = proposed;
268       If[MatrixRank[Append[basis, target]] == rank,
269         sol = Quiet[Check[LinearSolve[Transpose[basis], target], $Failed]];
270         If[sol === $Failed, Throw[err["LinearSolveFailed", "Exact subset-span solving failed."], tag]];
271         terms = MapThread[#1 AlgebraicNumber[theta, #2] &, {sol, basis}];
272         cert = VerifyRootDecomposition[a, terms, "Sum"];
273         If[FailureQ[cert], Throw[cert, tag]];
274         Throw[Join[cert, <|"Status" -> "GloballyOptimal",
275           "OptimalityReason" -> "CompleteConjugateSubsetTheorem",
276           "SplittingFieldDegree" -> dimension, "CandidateCount" -> candidateCount|>], tag]
277       ], {entry, entries}];
278   cert = VerifyRootDecomposition[a, {a}, "Sum"];
279   If[FailureQ[cert], cert, Join[cert, <|"Status" -> "GloballyOptimal",
280     "OptimalityReason" -> "CompleteConjugateSubsetTheorem",
281     "SplittingFieldDegree" -> dimension, "CandidateCount" -> candidateCount|>]],
282   tag]
283 ];
284 ];
285
286 End[];
287 EndPackage[];

```

B Independent verification design

The Python checks deliberately do not recognize algebraic numbers from floating-point approximations. Polynomial identities are reduced exactly. Real branches are checked by Sturm counts in rational intervals. For example, the cubic roots are isolated in

$$-\frac{683}{1000} < a < -\frac{682}{1000}, \quad -\frac{1325}{1000} < b < -\frac{1324}{1000}.$$

The degree-nine sum and product each have one real root, isolated respectively in

$$\left(-\frac{2008}{1000}, -\frac{2006}{1000}\right), \quad \left(\frac{902}{1000}, \frac{906}{1000}\right).$$

The reconstruction identities (12) and (13), not interval overlap alone, certify the equalities.

For the subset theorem, represent a trace-zero cubic root family by

$$r_1 = (1, 0), \quad r_2 = (0, 1), \quad r_3 = (-1, -1).$$

The conjugates of $a + b$ become $r_i \oplus r_j$ in a four-dimensional direct sum. The conjugates of ab become $r_i \otimes r_j$ in a four-dimensional tensor product. The two independent S_3 actions permute the indices. A subset sum's orbit size is its degree because these are faithful realizations of the relevant rational spans inside the splitting field. This avoids constructing a degree-36 primitive element just to verify the combinatorial and linear-algebraic part of the algorithm.

For the counterexample group, use pairs $(r, s) \in \mathbb{Z}/4\mathbb{Z} \times \mathbb{Z}/2\mathbb{Z}$ with multiplication

$$(r, s)(t, u) = (r + (-1)^s t, s + u).$$

Form two copies, then identify each pair with its product by the diagonal central involution. The script constructs the resulting 32 by 32 multiplication table, checks the center and exponent, and enumerates subgroups by adjoining generators until closure. Every computation uses integer or rational arithmetic.

References

- [1] V. Reshetnikov, *Factor a polynomial Root into Roots of smallest possible degree*, Mathematica Stack Exchange, question 105933, 2016; answers by Daniel Lichtblau and yarchik.
<https://mathematica.stackexchange.com/q/105933/7288>
- [2] Wolfram Research, *Root*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/Root.html>
- [3] Wolfram Research, *RootReduce*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/RootReduce.html>
- [4] Wolfram Research, *MinimalPolynomial*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/MinimalPolynomial.html>
- [5] Wolfram Research, *Resultant*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/Resultant.html>
- [6] Wolfram Research, *ToNumberField*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/ToNumberField.html>
- [7] Wolfram Research, *AlgebraicNumber*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/AlgebraicNumber.html>
- [8] Wolfram Research, *LinearSolve*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/LinearSolve.html>
- [9] Wolfram Research, *IrreduciblePolynomialQ*, Wolfram Language documentation.
<https://reference.wolfram.com/language/ref/IrreduciblePolynomialQ.html>
- [10] Wolfram Research, *Return*, Wolfram Language documentation, especially Possible Issues.
<https://reference.wolfram.com/language/ref/Return.html>
- [11] J. S. Milne, *Fields and Galois Theory*, course notes.
<https://www.jmilne.org/math/CourseNotes/FT.pdf>
- [12] C. Altamirano, *A Problem in Algebraic Number Theory*, MIT UROP+ final paper, August 31, 2017. Mentor: Atticus Christensen; project suggested by Bjorn Poonen. The additive descent is relevant; the product descent in Theorem 3.1 is contradicted by Section 8 of this article.
<https://math.mit.edu/research/undergraduate/urop-plus/documents/2017/Altamirano.pdf>
- [13] A.-S. Elsenhans and J. Klüners, *Computing subfields of number fields and applications to Galois group computations*, arXiv:1610.06837.
<https://arxiv.org/abs/1610.06837>